

Torch_npu & aclgraph



秦传瑜 - 华为
2025/09/20

🔥 目录

- 1. Torch_npu-基础功能其实很简单
- 2. Torch_npu-view类操作需要精细处理
- 3. Torch_npu- internal_format的支持方式
- 4. Torch_npu 与 pytorch彻底解耦
- 5. Torch_npu 在内存管理上的一点点创新
- 6. Torch_npu 的graph mode历程
- 7. Torch_npu 的fx对接
- 8. aclgraph 的基础设计原理
- 9. aclgraph的内存复用考虑

🔥 Torch_npu-基础功能其实很简单

起步于2019年的pytorch 1.0版本,

```
import torch
```

```
a = torch.Tensor([1])
```

```
a = a.to(“npu”) //申请device内存并拷贝
```

```
a.add(1) // 调用kernel执行
```

右图是修改的所有文件，可以分为两类：

1: pytorch/c10 下的，主要是流程控制类，由于原来的流程都已经存在，因此基本上是增加一个type就好

2: 实现对应的device/stream/memory资源对接

3: 在aten下，在native_functions.yaml里新增算子，并增加对应的Kernel调用（**难点是kernel如何提供，非CUDA生态构建好用算子体系还需时日。**）

pytorch/aten/src/ATen/CMakeLists.txt	+ 2	- 1
pytorch/aten/src/ATen/Context.cpp	+ 4	- 0
pytorch/aten/src/ATen/Context.h	+ 1	- 0
pytorch/aten/src/ATen/detail/NPUGuardImpl.cpp	+ 8	
pytorch/aten/src/ATen/detail/NPUGuardImpl.h	+ 61	
pytorch/aten/src/ATen/gen.py	+ 9	- 3
pytorch/aten/src/ATen/native/Copy.cpp	+ 1	- 1
pytorch/aten/src/ATen/native/DispatchStub.h	+ 17	- 0
pytorch/aten/src/ATen/native/native_functions.yaml	+ 13	- 0
pytorch/aten/src/ATen/native/npu/BinaryOpsKernel.cpp	+ 234	
pytorch/aten/src/ATen/native/npu/CopyKernel.cpp	+ 82	
pytorch/aten/src/ATen/native/npu/TensorFactories.cpp	+ 105	
pytorch/aten/src/ATen/native/npu/TensorFactories.h	+ 28	
pytorch/aten/src/ATen/native/npu/UnaryOpsKernel.cpp	+ 94	
pytorch/aten/src/ATen/preprocess_declarations.py	+ 1	- 1
pytorch/c10/CMakeLists.txt	+ 1	- 0
pytorch/c10/core/Backend.h	+ 24	- 1
pytorch/c10/core/Device.cpp	+ 2	- 1
pytorch/c10/core/DeviceType.cpp	+ 3	- 0
pytorch/c10/core/DeviceType.h	+ 3	- 1
pytorch/c10/core/TensorOptions.h	+ 2	- 0
pytorch/c10/core/TensorTypeId.cpp	+ 2	- 0
pytorch/c10/core/TensorTypeId.h	+ 1	- 1
pytorch/c10/npu/NPUAllocator.cpp	+ 130	
pytorch/c10/npu/NPUAllocator.h	+ 42	

🔥 Torch_npu-view类操作需要精细处理

https://docs.pytorch.org/docs/stable/tensor_view.html

Pytorch是base on CPU/NV的，对view类的行为不做实际内存搬移，而在执行时做非物理连续读。

与tensorflow等框架相比，tensor定义多了一个stride属性。

对主要是Scalar计算的CPU与SIMT友好，但是给基于SIMD的芯片出了一个难题。

实践证明，基于lazy contiguous的思路是性能最优解：

对view类Ops 保持只改变shape与stride的特点

在实际需要发生计算行为时前置 .contiguous()

这里的诀窍在于：在contiguous()还得尽量转为transpose等有专用指令支撑的kernel来保证SIMD场景高性能

但对于硬性地gather/scatter就没有太多的办法了。

🔥 Torch_npu- internal_format的支持方式

Pytorch提供基于数学语义的公共aten IR
但是芯片内部出于面效比考虑会存在与数学格式排布不一致的内部计算格式

理想情况下的芯片应该是尽量做好多pipeline掩盖数学格式与计算格式之间的转换
但是：如果芯片基于一些原因没能做到呢？此时就需要一种能携带计算格式的方案。

基于pytorch eager mode特点考虑，只能将这个信息放在tensor::storagempl里，
在pytorch1.8之前这个结构体是final的无法继承导致torch_npu无法独立于torch存在，在1.8之后可以继承storagempl实现。

```
c10 > core > C StorageImpl.h > {} c10 > StorageImpl > StorageImpl(caffe2::TypeMeta, int64_t, at::Allocator *, bool)
8 namespace c10 {
10 struct C10_API StorageImpl final : public c10::intrusive_ptr_target {
12     StorageImpl(
```

```
pytorch1.8adp > torch_npu > csrc > core > C NPUStorageImpl.h > {} torch_npu > NPUStorageImpl >
14 namespace torch_npu {
42 struct NPUStorageImpl : public c10::StorageImpl {
43     explicit NPUStorageImpl(use_byte_size_t use_byte_size,
44         size_t size_bytes,
45         at::DataPtr data_ptr,
46         at::Allocator* allocator,
47         bool resizable);
48     ~NPUStorageImpl() override = default;
49     // not private
50     NPUStorageDesc npu_desc_;
16 struct NPUStorageDesc {
17 public:
18     struct use_byte_size_t {};
19
20     c10::SmallVector<int64_t,5> base_sizes_;
21     c10::SmallVector<int64_t,5> base_strides_;
22     c10::SmallVector<int64_t,5> storage_sizes_;
23     int64_t base_offset_ = 0; // no use
24     use_byte_size_t base_dtype_; // no use
25     aclFormat origin_format_;
26     aclFormat npu_format_ = ACL_FORMAT_ND;
27     // used to make CANN GE tensor from storagImpl
28     caffe2::TypeMeta data_type_;
29 };
```

Torch_npu 与pytorch彻底解耦

<https://docs.pytorch.org/tutorials/advanced/privateuseone.html>

Other Modules

In addition to the above-mentioned parts, there are some other modules that can be expanded through `PrivateUse1`, such as `distributed collective communication`, `benchmark timer`, and others, which will be added in the future. One example about `PrivateUse1` integration is [Ascend NPU](#).

从pytorch1.0开始，到1.8最后一个storageImpl去除final后可以解耦，只剩下deviceId耦合为止，HW一直是羞羞答答的不敢联系社区增加deviceId，那几年一直处于过度自我审查约束的状态。

在2023年终于鼓起勇气再去提的时候，社区认为再增加已经不太合适（虽然我们提了之后又增加了几个DeviceID），给了一个privateuser1的方向，在密集提交代码解决了一些公共问题后，torch_npu终于与pytorch彻底解耦了。也算是HW为其他AI硬件厂商（尤其是国产）做的公共贡献了。

pytorch / c10 / core / Backend.h

```
Code Blame 216 lines (200 loc) · 5.49 KB
123 }
124
125 static inline Backend deviceTypeToBackend(DeviceType d) {
126     switch (d) {
127         case DeviceType::CPU:
128             return Backend::CPU;
129         case DeviceType::CUDA:
130             return Backend::CUDA;
131         case DeviceType::HIP:
132             return Backend::HIP;
133         default:
134             AT_ERROR("Unknown device type ", d);
135     }
```

```
pytorch1.8 > c10 > core > DeviceType.h > {} c10
13 namespace c10 {
14
15     enum class DeviceType : int8_t {
16         CPU = 0,
17         CUDA = 1, // CUDA.
18         MKLDNN = 2, // Reserved for explicit MKLDNN
19         OPENGL = 3, // OpenGL
20         OPENCL = 4, // OpenCL
21         IDEEP = 5, // IDEEP.
22         HIP = 6, // AMD HIP
23         FPGA = 7, // FPGA
24         MSNPU = 8, // MSNPU
25         XLA = 9, // XLA / TPU
26         Vulkan = 10, // Vulkan
27         Metal = 11, // Metal
28         XPU = 12, // XPU
29         // NB: If you add more devices:
30         // - Change the implementations of DeviceTy
31         // in DeviceType.cpp
32         // - Change the number below
33         COMPILE_TIME_MAX_DEVICE_TYPES = 13,
34     };
```

```
36 enum class DeviceType : int8_t {
37     CPU = 0,
38     CUDA = 1, // CUDA.
39     MKLDNN = 2, // Reserved for explicit MKLDNN
40     OPENGL = 3, // OpenGL
41     OPENCL = 4, // OpenCL
42     IDEEP = 5, // IDEEP.
43     HIP = 6, // AMD HIP
44     FPGA = 7, // FPGA
45     MAIA = 8, // ONNX Runtime / Microsoft
46     XLA = 9, // XLA / TPU
47     Vulkan = 10, // Vulkan
48     Metal = 11, // Metal
49     XPU = 12, // XPU
50     MPS = 13, // MPS
51     Meta = 14, // Meta (tensors with no data)
52     HPU = 15, // HPU / HABANA
53     VE = 16, // SX-Aurora / NEC
54     Lazy = 17, // Lazy Tensors
55     IPU = 18, // Graphcore IPU
56     MTIA = 19, // Meta training and inference devices
57     PrivateUse1 = 20, // PrivateUse1 device
```

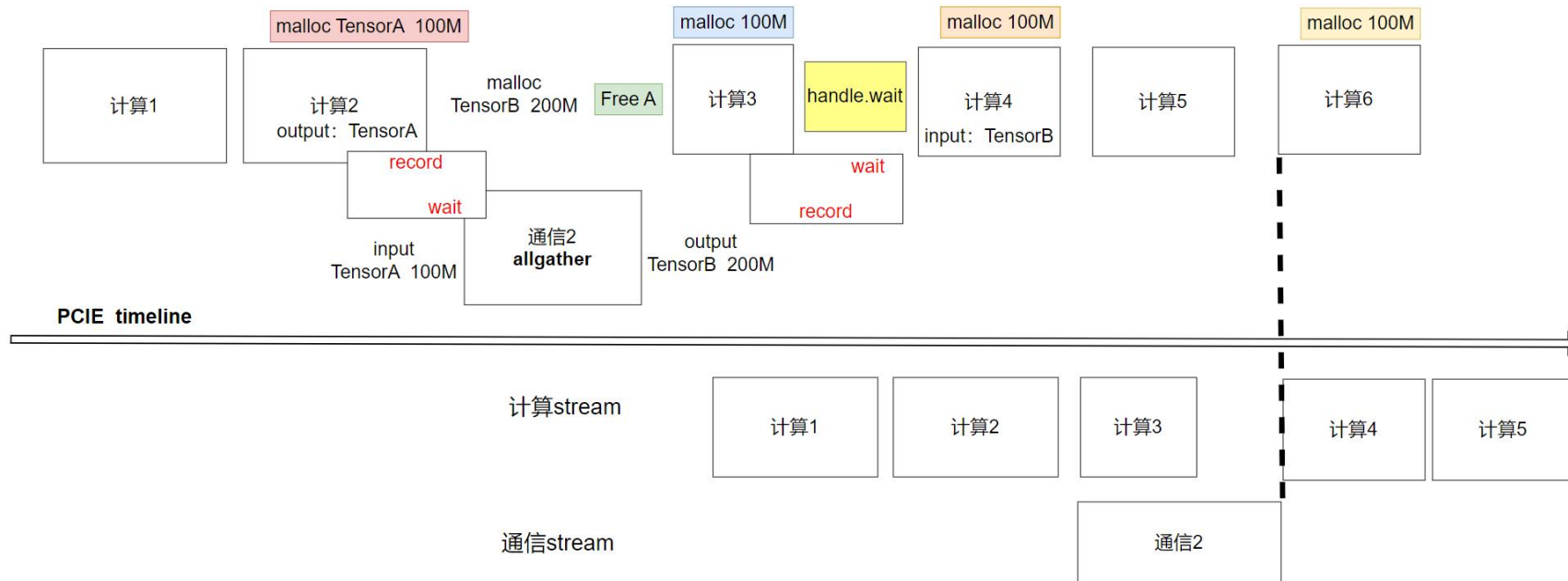
Torch_npu 在内存管理上的一点点创新

在2023年的大模型训练时，当时由于NPU上的内存64G相比NV 80G存在差距，因此绞尽脑汁做内存优化。

其中一个点是：在pytorch-npu上做了一个multi_stream_reuse的增强能力，基本逻辑是：因为tensorA对应的block会在生命周期结束时被做recordStream的操作标识上通信stream，那我们就可以在handle.wait()时做一个对等的eraseStream操作来去除这个通信stream的占用标识。在LLM训练场景，由于多流使用场景广泛存在，内存收益稳定10G以上。

主要收益来源：allocated / active / reserved 三种内存中，基本消除了active - allocated的占用，降低了reserved开销

示意图：原先计算6才能复用计算2申请的tensor，但是在优化后计算4就可以开始复用了。



社区在最新的2.8版本里进行了跟进：
<https://github.com/pytorch/pytorch/commit/35c45a4a315425a0940dcc040d8262b6331804dc>

🔥 Torch_npu 的graph mode历程

在pytorch1.x时代，pytorch社区只提供了并不好用的torch.script，转onnx也更多地是为了支持离线推理。

在2021年HW内部有一版基于lazy tensor原理的版本，
因为torch.script这种capture&replay机制unsound，在训练场景下不可用（推理场景可以人工check是否符合预期）
比torch_xla起步还要早一点，亲眼见证了torch_xla从torch.script转向lazy tensor方向的过程

当时的分析，基于trace机制的Pytorch的图有两个公共的问题难以解决：

- 1：控制流（for if 等逻辑无法入图）
- 2：同步导致的Device执行间隙（在新的图下发之前，Device只能等Pytorch本身Python框架构图）

因为lazy tensor本质上是Recapture & play机制，如果在device dispatch之后成图，头开销过大，尽管可以做各种缓存来缓解，但是判断是否是同一张图仍然需要一个model不带kernellaunch的forward流程执行开销。
导致：一旦有host device同步点存在，head overhead往往就基本消减掉了graph的融合收益。

在pytorch2.x时代，虽然问题1仍然存在，但是在dynamo里通过symbol及各种繁琐的操作生成了guard后有效地解决了问题2

因此在2023年就毫不犹豫地抛弃了lazy tensor版实现，直接基于fx.graph来进行graph相关后续操作。

🔥 Torch_npu 的fx对接

基于fx图的内存优化，时序调整等事情都是device无关的公共操作，基于社区开源工作就好。

从原理上来说，与pytorch框架无关的厂家graph能力只能起两个作用：

1：做kernel自动融合

2：做device kernel调度加速

第一点本质上考验的是：如何基于npu设备做kernel 的 codgen工作，基于inductor+Triton是当前pytorch社区在NV上的选择，但是这个选择对使用的用户感知并不大，如果有更优的实现可以替换掉。

第二点比较通用的方式就是做 capture & replay 机制：device task累计后一次性执行省去framework overhead。

当前在各家AI芯片厂家在第二点都比较成熟，第一点即使在NV上也呈现出勃勃生机之势，尚无最终定论。

🔥 Aclgraph 的基础设计原理

Cudagraph与torch_xla几乎是在同一时间出现。

Torch_xla是Google为了基于TensorFlow graph模式才能较好发挥性能的TPU做的无奈生态适配。（用户喜欢pytorch）

NV为了照顾庞大的cudnn/cublas/各种cuda算子的已有资产，推出了基于capture&replay的cudagraph机制。

aclGraph的原理与cudaGraph并无本质差异，因此能力及约束都是共通的，除了以下较少的差异点：

- 1：在设计实现过程中，无法理解为什么malloc memory一定不可以在global模式下工作，因此在NPU上并没有做约束。
由此导致的差异是：torch.npu.graph在capture时本质上并不需要warmup，而torch.cuda.graph需要
- 2：当前部分NPU的FA算子实现存在tensor shape固定但是actual_seq_len变化场景需要动态刷新tiling
（这个约束当前算子团队在尽力解除）
引申出一个对于已有aclgraph实例，需要动态刷新其中的部分node的诉求，对此aclgraph提供了graph_task_group_begin/graph_task_group_end, graph_task_update_begin/graph_task_update_end
通用的刷新部分node的API来应对此类诉求，原理是：通过再次基于新args的算子kernellaunch行为覆盖之前的task。
- 3：cudagraph是先在host上构图，然后在instance到device去，使用时存在host/device实例差异的感知，
aclgraph直接将capture过程中的task实例化到device上，简单直接。
（主要是没想清楚host/device双份管理的存在必要性是为啥）

Aclgraph的内存复用考虑

aclgraph的本质是对kernellaunch的task缓存后在device上反复replay，自然没有能力对task里的memory_addr做精准识别以及自动刷新，因此：如何基于这个地址不可变的特点适配好业务场景是上层业务场景对aclgraph的主要适配工作。

基于torch.npu.graph的eager mode使用需要精细考虑内存复用情况，基于torch.compile则省心很多，但是也有新的挑战。



Thanks

